

Tecnatura Superior en Desarrollo de Software — Material Complementario

Anexo 01A — Interfaces y Clases Abstractas en C#

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es una interfaz?

Una **interfaz** en C# es un contrato. Define qué métodos, propiedades o eventos debe tener una clase, pero no cómo los implementa. Es como un formulario en blanco: especifica los campos que deben completarse, pero no dice con qué valores.

Una clase que declara que implementa una interfaz se compromete a definir cada uno de los miembros declarados en ella. Si no lo hace, el compilador lo rechaza. Este mecanismo permite que distintas clases sean tratadas de forma uniforme por el código que las usa.

Definición precisa:

Una interfaz es un tipo de referencia que declara un conjunto de miembros sin implementación (por defecto). Cualquier clase o estructura que la implemente debe proveer una implementación concreta para cada miembro declarado.

Sintaxis básica

```
1 // convención: los nombres de interfaces comienzan con "I" mayúscula
2 public interface IPrinter
3 {
4     // declaración de método – sin cuerpo, sin modificador de acceso
5     void Print(string content);
6
7     // declaración de propiedad – solo la firma, sin implementación
8     bool IsOnline { get; }
```

```
9 }

```

Implementación de una interfaz

```
1 // la clase declara que implementa IPrinter usando ":"
2 public class LaserPrinter : IPrinter
3 {
4     // implementación concreta del método declarado en la interfaz
5     public void Print(string content)
6     {
7         // lógica específica de la impresora láser
8         Console.WriteLine($"Laser: {content}");
9     }
10
11     // implementación de la propiedad declarada en la interfaz
12     public bool IsOnline { get => true; }
13 }
```

Uso de la interfaz como tipo

La clave del polimorfismo con interfaces es que el código que usa `IPrinter` no sabe ni necesita saber qué clase concreta está usando. Esto permite sustituir implementaciones sin modificar el código que las consume.

```
1 // el tipo de la variable es la interfaz – no la clase concreta
2 IPrinter printer = new LaserPrinter();
3
4 // se llama al método de la interfaz – el compilador no sabe qué clase ejecuta
5 printer.Print("Hola mundo");
6
7 // mañana se puede cambiar por otra implementación sin tocar esta línea
8 // IPrinter printer = new InkjetPrinter(); → mismo código, distinto
   comportamiento

```

Analogía — El enchufe eléctrico

Un enchufe de pared define un contrato: dos patas de cierta forma, cierto voltaje. No importa si conectás un televisor, una computadora o un cargador de celular. Todos cumplen el mismo contrato (tienen el mismo tipo de ficha) y todos funcionan sin que la pared necesite saber qué aparato está conectado.

La pared es el código que usa la interfaz (`IPrinter`). El enchufe es la interfaz. El aparato que enchufás es la clase que la implementa (`LaserPrinter`).

Podés cambiar el aparato sin tocar la pared.

Mapeo al proyecto CommerX:

Pared (usa la interfaz) → CreateCustomerUseCase

Enchufe (la interfaz) → ICustomerRepository

Aparato (implementacion) → CustomerRepository (EF Core en Infraestructura)

Si mañana se cambia EF Core por Dapper, solo se reemplaza el "aparato".

El caso de uso no cambia una sola línea.

Características esenciales de las interfaces

Característica	Descripción
Sin estado	Una interfaz no puede tener campos (variables de instancia). Solo puede declarar propiedades, métodos y eventos.
Implementación múltiple	Una clase puede implementar varias interfaces a la vez. Esto es lo más cercano que C# tiene a la herencia múltiple.
Todos los miembros son públicos por defecto	En una interfaz no se escribe <code>public</code> — todos los miembros son implícitamente públicos.
No se instancia directamente	No se puede hacer <code>new IPrinter()</code> . Solo se instancia una clase que la implementa.
Puede extender otras interfaces	Una interfaz puede heredar de otra interfaz con <code>:</code> , acumulando miembros de ambas.

Implementación múltiple de interfaces

```
1 public interface IPrinter
2 {
3     void Print(string content);
4 }
5
6 public interface IScanner
7 {
8     string Scan();
9 }
10
11 // una clase puede implementar múltiples interfaces separadas por coma
12 public class MultifunctionDevice : IPrinter, IScanner
13 {
14     public void Print(string content) => Console.WriteLine(content);
15     public string Scan() => "documento escaneado";
16 }
```

Clases abstractas — cuándo usar una u otra

Las **clases abstractas** son similares a las interfaces en que no se pueden instanciar directamente, pero difieren en que pueden tener implementación parcial, estado (campos) y constructores. La decisión de cuándo usar una u otra es una de las más importantes del diseño orientado a objetos.

Sintaxis de clase abstracta

```
1 // abstract: no puede instanciarse – solo heredarse
2 public abstract class Shape
3 {
4     // puede tener campos – las interfaces no pueden
5     public string Color { get; set; }
6
7     // puede tener implementación concreta compartida
8     public void Describe() => Console.WriteLine($"Figura de color {Color}");
9
10    // abstract: las subclases DEBEN implementar este método
11    public abstract double Area();
12 }
```

```
1 // hereda de Shape y provee implementación de Area()
2 public class Circle : Shape
3 {
4     public double Radius { get; }
```

```

5
6 // override: implementa el método abstracto de la clase base
7 public override double Area() => Math.PI * Radius * Radius;
8 }

```

Interfaz vs clase abstracta — tabla comparativa

Aspecto	Interfaz	Clase abstracta
Puede tener campos	No	Si
Puede tener constructor	No	Si
Puede tener implementación	Solo métodos default (C# 8+)	Si — métodos concretos y abstractos
Herencia múltiple	Una clase puede implementar muchas	Solo se puede heredar de una
Cuándo usarla	Para definir un contrato de comportamiento entre tipos sin relación	Para compartir código y estructura entre tipos relacionados
Ejemplo en CommerX	ICustomerRepository , ICreateCustomerInputPort	DomainException , BaseEntity , ValueObject

Regla práctica:

Usá **interfaz** cuando querés definir un contrato que puede ser implementado por clases de jerarquías completamente distintas.

Usá **clase abstracta** cuando querés compartir implementación entre clases que tienen una relación "es un tipo de".

Errores comunes con interfaces

Error	Consecuencia
Poner modificadores de acceso en la	<code>public void Print()</code> dentro de una interfaz genera un error en versiones anteriores de C#. Los miembros son

Error	Consecuencia
interfaz	implícitamente públicos. En .NET 9 está permitido, pero es redundante.
Definir campos en la interfaz	Las interfaces no pueden tener campos. Si necesitas estado compartido, usá una clase abstracta.
Crear interfaces "gorillas"	Una interfaz con 20 métodos viola el Principio de Segregación de Interfaces (ISP). Cada interfaz debe tener un propósito específico y pocos miembros.
No usar la interfaz como tipo	Si declarás <code>LaserPrinter printer = new LaserPrinter()</code> en lugar de <code>IPrinter printer = new LaserPrinter()</code> , perdés el beneficio del polimorfismo.

Qué NO hacer

Antipatrón — Interfaz con demasiados métodos

```

1 // MAL: interfaz "gorda" — viola ISP
2 public interface ICustomerRepository
3 {
4     Task AddAsync(Customer c);
5     Task UpdateAsync(Customer c);
6     Task DeleteAsync(Guid id);
7     Task SendEmailAsync(string email); // MAL: no pertenece al repositorio
8     Task GenerateReportAsync(); // MAL: responsabilidad diferente
9 }

```

Antipatrón — Instanciar la clase concreta en lugar de usar la interfaz

```

1 // MAL: dependencia de la implementación concreta
2 public class CreateCustomerUseCase
3 {
4     // MAL: depende de la clase concreta — no de la interfaz
5     private readonly CustomerRepository _repository;
6
7     // BIEN: debería ser ICustomerRepository — no la clase concreta
8     private readonly ICustomerRepository _repository;
9 }

```



Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Tecnicatura Superior en Desarrollo de Software |
Asignatura: Programación II | Tema: Anexo 01A — Interfaces y Clases Abstractas en C# | Versión: 2026